TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU THỰC HÀNH

Học máy ứng dụng

Bài 3: Naive Bayes

Biên soạn ThS. Nguyễn Thái Anh

Năm học 2026 – 2027

Mục lục

1 Mục tiêu bài thực hành **2**
1.1 Chuẩn bị . . . 2

2 Định lý Bayes **2**
2.1 Một câu hỏi nên tự trả lời trước . . . 2
2.2 Giải bằng cách đếm người . . . 3
2.3 Công thức . . . 3
2.4 Tính bằng Python . . . 4

3 Biến văn bản thành số **5**
3.1 Bộ dữ liệu tin nhắn . . . 5
3.2 Túi từ . . . 6
3.3 Làm bằng scikit-learn . . . 6
3.4 Đọc kết quả . . . 8

4 Bộ lọc tin nhắn rác **8**
4.1 Công thức Naive Bayes . . . 9
4.2 Huấn luyện và chấm điểm . . . 9
4.3 Đọc kết quả . . . 11

5 Vấn đề số không và cách làm trơn **11**
5.1 Một số không đủ sức khoá cả phép nhân . . . 11
5.2 Làm trơn Laplace . . . 12
5.3 Tự cài đặt để thấy tận mắt . . . 12
5.4 Đọc kết quả . . . 15

6 Naive Bayes cho số liệu liên tục **15**
6.1 Đếm không còn dùng được . . . 15
6.2 So với hồi quy logistic trên cùng dữ liệu . . . 16
6.3 Đọc kết quả . . . 18
6.4 Chỗ hai mô hình khác nhau rõ rệt . . . 18

7 Bài tập **19**

8 Phụ lục A, bảng tra nhanh **21**

9 Phụ lục B, lỗi thường gặp **22**

1. Mục tiêu bài thực hành

Hai buổi trước, máy học bằng cách vẽ một ranh giới rồi dò cho ranh giới đó ngày càng khớp. Buổi này máy làm việc khác hẳn: nó mô tả xem mỗi lớp trông như thế nào, rồi hỏi lớp nào dễ sinh ra mẫu đang xét hơn.

Sau khi hoàn thành bài này, sinh viên có khả năng:

1. Tính được xác suất hậu nghiệm bằng định lý Bayes, cả bằng công thức lẫn bằng cách đếm.
2. Giải thích được vì sao một xét nghiệm chính xác 99 phần trăm vẫn có thể báo nhầm quá nửa số ca.
3. Biến một tập văn bản thành bảng số bằng túi từ và `CountVectorizer`.
4. Huấn luyện bộ lọc tin nhắn rác bằng `MultinomialNB` rồi chấm điểm nó.
5. Chỉ ra được chỗ mô hình gãy khi bỏ làm trơn Laplace, bằng cách tự cài đặt lại.
6. Dùng `GaussianNB` cho số liệu liên tục và so sánh với hồi quy logistic của bài 2.

Hai bộ dữ liệu

Bài này dùng `data/tin_nhan.csv` gồm 300 tin nhắn, và dùng lại `data/sinh_vien.csv` của bài 2. Dùng lại bộ cũ là có chủ ý, vì như vậy mới so sánh công bằng được hai mô hình trên cùng một đề bài.

1.1. Chuẩn bị

Vẫn môi trường của bài 1, không cài thêm gì.

```
cd C:\MayHoc
hocmay\Scripts\activate
cd bai03_naive_bayes
```

Bảng 1: *Năm tệp mã của buổi thực hành, chạy theo đúng thứ tự này.*

Mục	Tệp	Nội dung
2	`d1_bayes_tay.py`	Tính định lý Bayes bằng tay
3	`d2_tui_tu.py`	Biến văn bản thành bảng đếm từ
4	`d3_loc_thu_rac.py`	Huấn luyện và chấm điểm bộ lọc
5	`d4_lam_tron.py`	Tự cài Naive Bayes để thấy vấn đề số không
6	`d5_gaussian.py`	Naive Bayes cho số liệu liên tục

2. Định lý Bayes

2.1. Một câu hỏi nên tự trả lời trước

Một căn bệnh có ở 1 phần trăm dân số. Xét nghiệm rất tốt: người có bệnh thì 99 phần trăm cho kết quả dương tính, người không bệnh thì 99 phần trăm cho kết quả âm tính.

Sinh viên nhận kết quả dương tính. Khả năng có bệnh thật là bao nhiêu?

Hầu hết mọi người trả lời khoảng 99 phần trăm. Đáp án đúng là **0.5**, tức chỉ năm mươi năm mươi.

2.2. Giải bằng cách đếm người

Cách dễ hiểu nhất là bỏ hẳn công thức đi và đếm trên một nhóm 10000 người.

Bảng 2: *Mười nghìn người đi xét nghiệm, chia theo bệnh thật và kết quả xét nghiệm.*

	Dương tính	**Âm tính**	**Tổng**
Có bệnh	99	1	100
Không bệnh	99	9801	9900
Tổng	**198**	9802	10000

Nhìn cột dương tính: có 198 người, trong đó chỉ 99 người bệnh thật. Vậy tỷ lệ là $99/198 = 0.5$.

Vì sao trực giác sai

Vì bệnh quá hiếm. Nhóm không bệnh đông gấp 99 lần nhóm có bệnh, nên dù chỉ sai 1 phần trăm, nhóm đó vẫn tạo ra đúng 99 ca dương tính giả. Con số 99 đó bằng chằn chặn số ca dương tính thật, nên hai nhóm hoà nhau.

2.3. Công thức

Định lý Bayes

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(B \mid A)\,\mathbb{P}(A)}{\mathbb{P}(B)}$$

Bốn thành phần có tên riêng, và nhớ tên thì dễ dùng hơn nhiều.

Bảng 3: *Tên gọi của bốn thành phần trong định lý Bayes.*

Thành phần	**Tên gọi**	**Trong ví dụ xét nghiệm**
$\mathbb{P}(A)$	Tiên nghiệm *(prior)*	Tỷ lệ mắc bệnh, bằng 0.01
$\mathbb{P}(B \mid A)$	Khả năng *(likelihood)*	Độ nhạy của xét nghiệm, bằng 0.99
$\mathbb{P}(B)$	Bằng chứng *(evidence)*	Tỷ lệ dương tính chung, bằng 0.0198
$\mathbb{P}(A \mid B)$	Hậu nghiệm *(posterior)*	Điều ta muốn biết, bằng 0.5

Lỗi tư duy phổ biến nhất

Đảo hai vế của gạch đứng rồi tưởng giá trị không đổi.
$\mathbb{P}(\text{dương} \mid \text{bệnh}) = 0.99$ nhưng $\mathbb{P}(\text{bệnh} \mid \text{dương}) = 0.5$. Hai con số cách nhau rất xa. Đây là sai lầm hay gặp cả trong y tế, toà án lẫn báo chí, và định lý Bayes chính là công cụ đổi chiều cho đúng.

2.4. Tính bằng Python

```python
# -*- coding: utf-8 -*-
"""Bước 1: tính định lý Bayes bằng tay, không cần thư viện học máy.

Chú thích viết tiếng Việt có dấu, phần in ra màn hình viết không dấu.
"""


def bayes(tien_nghiem, kha_nang, kha_nang_nguoc):
    """Tra ve xac suat hau nghiem.

    tien_nghiem      : P(A), niem tin truoc khi co bang chung
    kha_nang         : P(B | A), bang chung xuat hien khi A dung
    kha_nang_nguoc   : P(B | khong A), bang chung xuat hien khi A sai
    """
    tu_so = kha_nang * tien_nghiem
    mau_so = tu_so + kha_nang_nguoc * (1 - tien_nghiem)
    return tu_so / mau_so


print("Vi du 1: xet nghiem benh")
print("  Ty le mac benh 1 phan tram, xet nghiem dung 99 phan tram.")
p = bayes(tien_nghiem=0.01, kha_nang=0.99, kha_nang_nguoc=0.01)
print(f"  Duong tinh thi kha nang co benh that = {p:.4f}")
print()

# Giữ nguyên chất lượng xét nghiệm, chỉ đổi tỷ lệ mắc bệnh
print("Cung xet nghiem do, doi ty le mac benh:")
print("  Ty le mac benh    Hau nghiem khi duong tinh")
for ty_le in [0.001, 0.01, 0.05, 0.10, 0.50]:
    print(f"    {ty_le:6.3f}              {bayes(ty_le, 0.99, 0.01):.4f}")
print()

print("Vi du 2: hai day chuyen san xuat")
print("  Day A lam 80 phan tram san pham, ty le loi 1 phan tram.")
print("  Day B lam 20 phan tram san pham, ty le loi 5 phan tram.")
# Ở đây "A đúng" nghĩa là sản phẩm đến từ dây B
p_b = bayes(tien_nghiem=0.20, kha_nang=0.05, kha_nang_nguoc=0.01)
print(f"  Nhat duoc san pham loi, kha nang no tu day B = {p_b:.4f}")
print()

print("Kiem lai bang cach dem tren 1000 san pham:")
loi_a = 800 * 0.01
loi_b = 200 * 0.05
print(f"  Day A: 800 san pham, {loi_a:.0f} cai loi")
print(f"  Day B: 200 san pham, {loi_b:.0f} cai loi")
tong_loi = loi_a + loi_b
print(f"  Ty le tu day B = {loi_b} / {tong_loi} = {loi_b / tong_loi:.4f}")
```

Hàm `bayes` nhận ba con số và trả về hậu nghiệm. Điểm đáng chú ý là mẫu số không phải nhập vào mà được dựng từ hai nhánh: nhánh A đúng và nhánh A sai. Cách viết này gọi là công thức xác suất toàn phần, và nó giúp ta không phải tự tính $\mathbb{P}(B)$.

```
python code/d1_bayes_tay.py
```

```
Vi du 1: xet nghiem benh
  Ty le mac benh 1 phan tram, xet nghiem dung 99 phan tram.
  Duong tinh thi kha nang co benh that = 0.5000

Cung xet nghiem do, doi ty le mac benh:
  Ty le mac benh    Hau nghiem khi duong tinh
     0.001              0.0902
     0.010              0.5000
     0.050              0.8390
     0.100              0.9167
     0.500              0.9900

Vi du 2: hai day chuyen san xuat
  Day A lam 80 phan tram san pham, ty le loi 1 phan tram.
  Day B lam 20 phan tram san pham, ty le loi 5 phan tram.
  Nhat duoc san pham loi, kha nang no tu day B = 0.5556

Kiem lai bang cach dem tren 1000 san pham:
  Day A: 800 san pham, 8 cai loi
  Day B: 200 san pham, 10 cai loi
  Ty le tu day B = 10.0 / 18.0 = 0.5556
```

Bảng giữa là phần đáng suy nghĩ nhất. Chất lượng xét nghiệm không đổi chút nào, chỉ có tỷ lệ mắc bệnh đổi, mà hậu nghiệm chạy từ 0.0902 lên tới 0.9900.

Bài học mang sang phần sau

Xác suất tiên nghiệm quan trọng ngang với bằng chứng. Trong phân loại, tiên nghiệm chính là tỷ lệ của từng lớp trong dữ liệu huấn luyện, và Naive Bayes luôn nhân nó vào. Đó là lý do một lớp hiếm rất khó được mô hình chọn.

3. Biến văn bản thành số

3.1. Bộ dữ liệu tin nhắn

Tệp `data/tin_nhan.csv` có 300 dòng và 2 cột. Cột `noi_dung` là chữ, cột `la_rac` là nhãn với 1 nghĩa là tin rác và 0 nghĩa là tin thường.

Về bộ số liệu

Đây là số liệu mô phỏng dùng cho mục đích giảng dạy. Tin nhắn được ghép từ các nhóm từ đặt sẵn, và có khoảng một phần năm tin rác cố tình viết như tin thường. Nếu hai lớp tách bạch hoàn toàn thì mô hình nào cũng đạt 100 phần trăm và buổi học không còn gì để bàn.

3.2. Túi từ

Máy không đọc được chữ, nên việc đầu tiên là đổi mỗi tin nhắn thành một dãy số. Cách đơn giản nhất là **túi từ** *(bag of words)*: lập một từ điển gồm mọi từ xuất hiện trong kho, rồi mỗi tin nhắn thành một dòng đếm số lần từng từ xuất hiện.

Bảng 4: *Ví dụ túi từ với một từ điển bốn từ.*

Tin nhắn	trúng	thưởng	họp	nhóm	Nhãn
trúng thưởng trúng ngay	2	1	0	0	rác
họp nhóm chiều mai	0	0	1	1	thường
nhóm mình trúng thưởng	1	1	0	1	rác

Túi từ bỏ mất thứ tự từ

Câu *chó cắn người* và câu *người cắn chó* cho ra đúng một túi từ giống nhau. Với bài lọc tin rác thì mất mát này chấp nhận được, vì bản thân từ ngữ đã đủ tố cáo. Với bài phân tích cảm xúc hay dịch máy thì thứ tự quan trọng hơn nhiều, và túi từ không còn đủ.

3.3. Làm bằng scikit-learn

```python
# -*- coding: utf-8 -*-
"""Bước 2: đọc kho tin nhắn và biến văn bản thành bảng số."""

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

df = pd.read_csv("data/tin_nhan.csv")

print("Kich thuoc bang:", df.shape)
print()
print("So tin theo nhan (1 la rac, 0 la thuong):")
print(df["la_rac"].value_counts())
print()
print("Ty le tin rac:", round(df["la_rac"].mean(), 4))
print()

print("Ba tin nhan dau tien:")
for i in range(3):
    print(f"  [{df.la_rac[i]}] {df.noi_dung[i]}")
print()

# token_pattern mặc định của scikit-learn bỏ mất từ một ký tự,
# nên ta chỉ định tách từ bằng khoảng trắng cho đơn giản.
vec = CountVectorizer(token_pattern=r"(?u)\{}S+")
X = vec.fit_transform(df["noi_dung"])

print("Kich thuoc bang dem tu:", X.shape)
print("So tu trong tu dien:", len(vec.vocabulary_))
print()
```

```
30
31 # Đếm tổng số lần xuất hiện của từng từ, tách theo hai lớp
32 dem = pd.DataFrame(X.toarray(), columns=vec.get_feature_names_out())
33 dem["la_rac"] = df["la_rac"]
34 tong = dem.groupby("la_rac").sum().T
35 tong.columns = ["thuong", "rac"]
36
37 print("Muoi tu xuat hien nhieu nhat trong tin rac:")
38 print(tong.sort_values("rac", ascending=False).head(10))
39 print()
40 print("Muoi tu xuat hien nhieu nhat trong tin thuong:")
41 print(tong.sort_values("thuong", ascending=False).head(10))
```

Có một tham số cần chú ý. Mặc định `CountVectorizer` bỏ qua mọi từ chỉ có một ký tự, và nó cũng cắt chữ theo quy tắc riêng. Với tiếng Việt, ta chỉ định `token_pattern=r"(?u)\S+"` để tách từ đơn giản bằng khoảng trắng.

```
python code/d2_tui_tu.py
```

```
Kich thuoc bang: (300, 2)

So tin theo nhan (1 la rac, 0 la thuong):
la_rac
0    180
1    120
Name: count, dtype: int64

Ty le tin rac: 0.4

Ba tin nhan dau tien:
  [0] cáo gửi cáo thầy nhóm cáo gửi có cho
  [1] miễn giảm ngay hạn miễn tay nhé thưởng cho nhanh
  [0] cho bạn mai gửi nhóm bài chiều

Kich thuoc bang dem tu: (300, 40)
So tu trong tu dien: 40

Muoi tu xuat hien nhieu nhat trong tin rac:
       thuong  rac
tay         3   48
ngay        4   47
nhanh       9   44
chót        6   43
giảm        6   42
lúc        39   42
quà         8   41
click       5   40
phí         6   40
miễn        4   39
```

```
Muoi tu xuat hien nhieu nhat trong tin thuong:
       thuong  rac
thầy       74    7
bài        73    4
cáo        72    8
chiều      68    8
ạ          65    7
tài        64   11
tập        63    6
gửi        61   10
lớp        60    2
nhóm       58    7
```

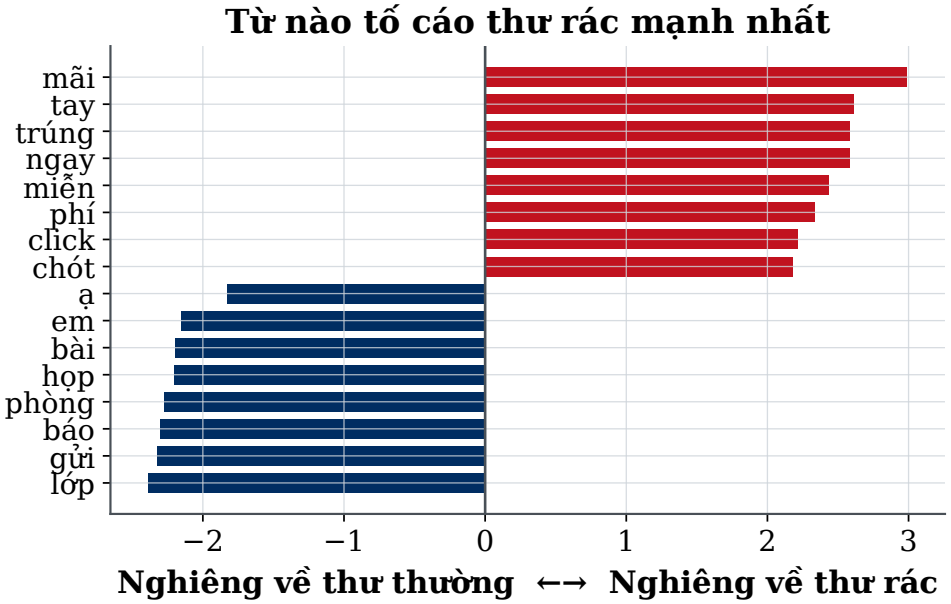
3.4. Đọc kết quả

Bảng đếm có kích thước 300 nhân 40, nghĩa là 300 tin nhắn và một từ điển 40 từ. Mỗi tin nhắn giờ là một dãy 40 con số, và máy làm việc được với nó.

Hai bảng cuối cho thấy điều quan trọng nhất. Các từ như `tay`, `ngay`, `nhanh`, `click` xuất hiện rất nhiều trong tin rác nhưng gần như vắng bóng ở tin thường. Ngược lại các từ như `báo`, `cáo`, `thầy` chỉ thấy ở tin thường.

Một chi tiết đáng để ý

Từ `lúc` xuất hiện 39 lần ở tin thường và 42 lần ở tin rác, gần như bằng nhau. Từ như vậy **không mang thông tin phân loại**, nó chỉ làm nền. Naive Bayes tự xử lý được chuyện này, vì xác suất của nó gần bằng nhau ở hai lớp nên nhân vào không làm lệch bên nào.



Hình 1: *Mức chênh lệch giữa hai lớp của từng từ. Từ càng lệch sang phải càng tố cáo tin rác mạnh.*

4. Bộ lọc tin nhắn rác

4.1. Công thức Naive Bayes

Với một tin nhắn gồm các từ $x_1, \ldots, x_d$, ta muốn biết lớp nào có xác suất hậu nghiệm lớn hơn. Định lý Bayes cho

Quy tắc quyết định

$$\hat{y} = \arg\max_{k} \; \mathbb{P}(y = k) \prod_{j=1}^{d} \mathbb{P}(x_j \mid y = k)$$

Mẫu số của định lý Bayes đã được bỏ đi, vì nó giống hệt nhau ở mọi lớp nên không ảnh hưởng tới việc lớp nào lớn hơn.

Dấu tích lớn xuất hiện được là nhờ **giả định ngây thơ**: khi đã biết lớp, các từ được coi như không liên quan gì tới nhau. Giả định này sai, vì trong tin rác từ *trúng* và từ *thưởng* rất hay đi cùng nhau. Nhưng thiếu nó thì ta phải ước lượng xác suất của từng tổ hợp từ, và với 40 từ thì số tổ hợp đã hơn một nghìn tỷ.

Toàn bộ việc huấn luyện chỉ là đếm

$$\mathbb{P}(\text{từ } j \mid \text{lớp } k) = \frac{\text{số lần từ } j \text{ xuất hiện trong lớp } k}{\text{tổng số từ của lớp } k}$$

Không có vòng lặp dò tìm nào như bài 2. Máy duyệt dữ liệu đúng một lượt rồi xong. Đây là lý do Naive Bayes từng là lựa chọn mặc định cho lọc thư rác suốt nhiều năm.

4.2. Huấn luyện và chấm điểm

```python
# -*- coding: utf-8 -*-
"""Bước 3: huấn luyện bộ lọc thư rác và chấm điểm nó."""

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             precision_score, recall_score)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

df = pd.read_csv("data/tin_nhan.csv")
vec = CountVectorizer(token_pattern=r"(?u)\{}S+")
X = vec.fit_transform(df["noi_dung"])
y = df["la_rac"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)
print("So tin de hoc     :", X_train.shape[0])
print("So tin de kiem tra:", X_test.shape[0])
print()

mo_hinh = MultinomialNB(alpha=1.0)
```

```
mo_hinh.fit(X_train, y_train)
y_pred = mo_hinh.predict(X_test)

tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
print("Ma tran nham lan")
print(f"  TN = {tn:2d}   thu thuong duoc giu dung")
print(f"  FP = {fp:2d}   thu thuong bi xoa nham")
print(f"  FN = {fn:2d}   thu rac lot luoi")
print(f"  TP = {tp:2d}   thu rac bi chan dung")
print()

print(f"Accuracy  = {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision = {precision_score(y_test, y_pred):.4f}")
print(f"Recall    = {recall_score(y_test, y_pred):.4f}")
print()

# Thử bộ lọc trên vài tin nhắn tự gõ
tin_moi = [
    "trung thuong khuyen mai nhanh tay click ngay",
    "hop nhom bao cao chieu mai phong 305",
    "ban gui giup em tai lieu bai tap nhe",
]
X_moi = vec.transform(tin_moi)
p_rac = mo_hinh.predict_proba(X_moi)[:, 1]

print("Thu tren ba tin nhan tu go:")
for tin, p in zip(tin_moi, p_rac):
    nhan = "RAC" if p >= 0.5 else "thuong"
    print(f"  [{nhan:6s}] p = {p:.4f}  <- {tin}")
```

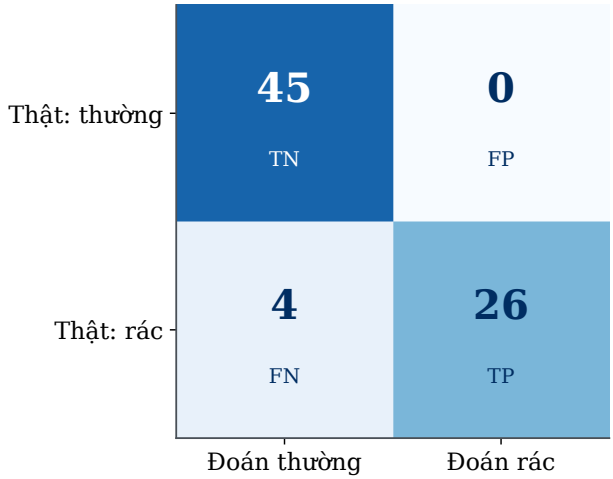
```
python code/d3_loc_thu_rac.py
```

```
So tin de hoc     : 225
So tin de kiem tra: 75

Ma tran nham lan
  TN = 45   thu thuong duoc giu dung
  FP =  0   thu thuong bi xoa nham
  FN =  4   thu rac lot luoi
  TP = 26   thu rac bi chan dung

Accuracy  = 0.9467
Precision = 1.0000
Recall    = 0.8667

Thu tren ba tin nhan tu go:
  [RAC   ] p = 0.9992  <- trung thuong khuyen mai nhanh tay click ngay
  [thuong] p = 0.0983  <- hop nhom bao cao chieu mai phong 305
  [thuong] p = 0.0719  <- ban gui giup em tai lieu bai tap nhe
```



Hình 2: *Ma trận nhầm lẫn của bộ lọc trên 75 tin nhắn kiểm tra.*

4.3. Đọc kết quả

Accuracy đạt 0.9467, nhưng con số đáng chú ý hơn nằm ở hai cột còn lại.

- **Precision bằng 1.0000**. Trong tất cả những tin mà bộ lọc gán nhãn rác, không có tin nào là thư thật. Nghĩa là FP bằng 0.
- **Recall bằng 0.8667**. Trong 30 tin rác của tập kiểm tra, bộ lọc bắt được 26 tin và để lọt 4 tin.

Vì sao kết quả này đáng mừng

Nhớ lại bài 2, phần chọn ngưỡng. Với bộ lọc thư rác, sai kiểu FP nghĩa là xoá nhầm một lá thư thật và lá thư đó mất luôn, hậu quả rất nặng. Sai kiểu FN chỉ là để lọt một tin rác vào hộp thư, người dùng tự xoá tay, hậu quả nhẹ.
Bộ lọc này để lọt 4 tin rác nhưng không xoá nhầm lá thư nào. Đó đúng là kiểu sai rẻ tiền mà ta muốn, và nó quan trọng hơn con số accuracy.

Phần cuối thử bộ lọc trên ba tin nhắn tự gõ. Tin quảng cáo nhận xác suất 0.9992, hai tin thường nhận 0.0983 và 0.0719. Mô hình rất dứt khoát, và đó vừa là ưu điểm vừa là điều phải cảnh giác.

Xác suất của Naive Bayes thường quá tự tin

Vì giả định ngây thơ sai, mô hình **đếm cùng một bằng chứng nhiều lần** khi các từ đi kèm nhau. Hệ quả là con số xác suất bị đẩy sát 0 hoặc sát 1.
Dùng nó để *chọn lớp* thì tốt. Dùng nó làm *xác suất thật* để đặt ngưỡng như bài 2 thì phải rất cẩn thận, vì một con số 0.9992 ở đây không có nghĩa là mô hình chắc chắn tới 99.92 phần trăm.

5. Vấn đề số không và cách làm trơn

5.1. Một số không đủ sức khoá cả phép nhân

Công thức Naive Bayes là một phép **nhân** dài. Trong một phép nhân, chỉ cần một thừa số bằng 0 là cả tích bằng 0, bất kể các thừa số còn lại lớn tới đâu.

Chuyện đó xảy ra khi nào? Khi có một từ chưa từng xuất hiện trong một lớp nào đó của tập huấn luyện. Lúc ấy số đếm bằng 0, nên xác suất của từ đó cũng bằng 0.

Hậu quả

Một tin nhắn chứa từ *trúng* và từ *thưởng*, tức hai dấu hiệu rác rất mạnh, vẫn có thể bị xếp vào lớp thường chỉ vì nó cũng chứa một từ mà lớp rác chưa từng gặp. Mô hình bị một từ duy nhất khoá miệng.

5.2. Làm trơn Laplace

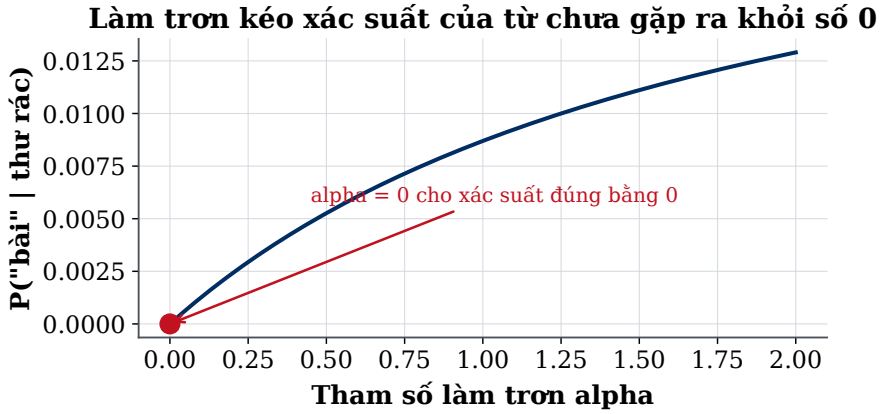
Cách chữa đơn giản tới bất ngờ: cộng thêm một lượng nhỏ vào mọi số đếm.

Ước lượng có làm trơn

$$\mathbb{P}(\text{từ } j \mid \text{lớp } k) = \frac{\text{đếm}_{jk} + \alpha}{\text{tổng}_k + \alpha V}$$

V là số từ trong từ điển, còn α thường chọn bằng 1.

Coi như mỗi từ đã xuất hiện sẵn α lần ở mọi lớp trước khi ta nhìn vào dữ liệu. Nhờ vậy không xác suất nào bằng 0 nữa. Mẫu số phải cộng αV để tổng các xác suất vẫn bằng 1.



Hình 3: *Xác suất của một từ chưa từng gặp ở lớp rác, vẽ theo tham số làm trơn. Tại alpha bằng 0, xác suất đúng bằng 0.*

5.3. Tự cài đặt để thấy tận mắt

Thư viện đã làm trơn sẵn với `alpha=1.0`, nên nếu chỉ gọi thư viện thì sinh viên không bao giờ thấy chỗ hỏng. Tệp dưới đây tự cài Naive Bayes trên một kho tí hon gồm sáu tin nhắn, để bật hoặc tắt làm trơn tuỳ ý.

```
# -*- coding: utf-8 -*-
"""Bước 4: tự cài Naive Bayes trên một kho nhỏ để thấy vấn đề số không.

Đoạn này cố ý KHÔNG dùng scikit-learn, vì thư viện đã làm trơn sẵn nên
sinh viên không nhìn thấy chỗ hỏng.
"""

from collections import Counter

# Kho tin nhắn tí hon, viết thẳng vào mã cho dễ theo dõi
```

```
kho = [
    ("trung thuong ngay", 1),
    ("khuyen mai giam gia", 1),
    ("trung thuong click", 1),
    ("hop nhom chieu mai", 0),
    ("bai tap nhom nop mai", 0),
    ("thay gui tai lieu", 0),
]

tu_dien = sorted({t for cau, _ in kho for t in cau.split()})
V = len(tu_dien)
print(f"Tu dien co {V} tu:")
# In thành nhiều dòng, mỗi dòng 9 từ, để không tràn ra ngoài khung
for i in range(0, V, 9):
    print("  " + " ".join(tu_dien[i:i + 9]))
print()

# Đếm số lần mỗi từ xuất hiện trong từng lớp
dem = {0: Counter(), 1: Counter()}
tong = {0: 0, 1: 0}
for cau, nhan in kho:
    for t in cau.split():
        dem[nhan][t] += 1
        tong[nhan] += 1

print("So lan xuat hien cua tung tu")
print(f"  {'tu':12s} {'rac':>5s} {'thuong':>7s}")
for t in tu_dien:
    print(f"  {t:12s} {dem[1][t]:5d} {dem[0][t]:7d}")
print()


def xac_suat(tu, nhan, alpha):
    """P(tu | nhan) co lam tron Laplace voi he so alpha."""
    return (dem[nhan][tu] + alpha) / (tong[nhan] + alpha * V)


def diem(cau, nhan, alpha):
    """Tich cua xac suat tien nghiem va cac xac suat tu."""
    p = 0.5          # hai lop can bang nen tien nghiem bang nhau
    for t in cau.split():
        if t in tu_dien:
            p *= xac_suat(t, nhan, alpha)
    return p


cau_thu = "trung thuong bai tap"
print(f"Tin nhan thu: \{}"{cau_thu}\{}"")
print("Tu 'bai' chi xuat hien o lop thuong, chua bao gio o lop rac.")
```

```
print()

for alpha in [0.0, 1.0]:
    d_rac = diem(cau_thu, 1, alpha)
    d_thuong = diem(cau_thu, 0, alpha)
    ket = "rac" if d_rac > d_thuong else "thuong"
    print(f"alpha = {alpha}")
    print(f"  diem lop rac    = {d_rac:.10f}")
    print(f"  diem lop thuong = {d_thuong:.10f}")
    print(f"  ket luan        = {ket}")
    print()

print("Khong lam tron, CA HAI diem deu bang 0 nen mo hinh te liet.")
print("Lop rac bang 0 vi tu 'bai' chua tung xuat hien o lop rac.")
print("Lop thuong bang 0 vi tu 'trung' chua tung xuat hien o lop thuong.")
print("Co lam tron, diem lop rac lon hon va mo hinh ket luan dung.")
```

```
python code/d4_lam_tron.py
```

```
Tu dien co 18 tu:
  bai chieu click gia giam gui hop khuyen lieu
  mai ngay nhom nop tai tap thay thuong trung

So lan xuat hien cua tung tu
  tu             rac  thuong
  bai              0       1
  chieu            0       1
  click            1       0
  gia              1       0
  giam             1       0
  gui              0       1
  hop              0       1
  khuyen           1       0
  lieu             0       1
  mai              1       2
  ngay             1       0
  nhom             0       2
  nop              0       1
  tai              0       1
  tap              0       1
  thay             0       1
  thuong           2       0
  trung            2       0

Tin nhan thu: "trung thuong bai tap"
Tu 'bai' chi xuat hien o lop thuong, chua bao gio o lop rac.

alpha = 0.0
  diem lop rac    = 0.0000000000
```

```
  diem lop thuong = 0.0000000000
  ket luan        = thuong

alpha = 1.0
  diem lop rac    = 0.0000073212
  diem lop thuong = 0.0000021656
  ket luan        = rac

Khong lam tron, CA HAI diem deu bang 0 nen mo hinh te liet.
Lop rac bang 0 vi tu 'bai' chua tung xuat hien o lop rac.
Lop thuong bang 0 vi tu 'trung' chua tung xuat hien o lop thuong.
Co lam tron, diem lop rac lon hon va mo hinh ket luan dung.
```

5.4. Đọc kết quả

Tin nhắn thử là *trúng thưởng bài tập*, một tin rác có lẫn hai từ của lớp thường.

Khi $\alpha = 0$, kết quả còn tệ hơn dự đoán: **cả hai điểm đều bằng 0**. Lớp rác bằng 0 vì từ `bài` chưa từng xuất hiện ở lớp rác. Lớp thường bằng 0 vì từ `trúng` chưa từng xuất hiện ở lớp thường. Mô hình tê liệt hoàn toàn và phải chọn bừa.

Khi $\alpha = 1$, điểm lớp rác bằng 0.0000073212 còn lớp thường bằng 0.0000021656. Cả hai đều rất nhỏ nhưng khác 0, nên so sánh được, và mô hình kết luận đúng là tin rác.

Vì sao hai con số nhỏ tới vậy

Vì chúng là tích của bốn xác suất, mỗi cái cỡ vài phần trăm. Với tin nhắn dài hơn, tích còn nhỏ nữa và máy tính sẽ làm tròn thành đúng số 0, hiện tượng gọi là tràn dưới *(underflow)*. Cách chữa là lấy loga rồi cộng thay vì nhân:

$$\log\left(\mathbb{P}(y)\prod_j \mathbb{P}(x_j \mid y)\right) = \log\mathbb{P}(y) + \sum_j \log\mathbb{P}(x_j \mid y)$$

Hàm loga đồng biến nên thứ tự hai lớp không đổi. Scikit-learn làm sẵn mẹo này, và thuộc tính `feature_log_prob_` chính là các giá trị loga đó.

Làm trơn không cứu được từ hoàn toàn mới

Làm trơn chỉ xử lý được từ *có trong từ điển nhưng chưa gặp ở một lớp*.
Một từ chưa từng xuất hiện trong kho huấn luyện thì không có trong từ điển, nên `CountVectorizer` bỏ qua nó hoàn toàn. Đó là lý do bộ lọc thư rác phải được huấn luyện lại định kỳ khi kẻ gửi rác đổi từ ngữ. Hiện tượng dữ liệu đổi theo thời gian gọi là trôi dữ liệu *(data drift)*.

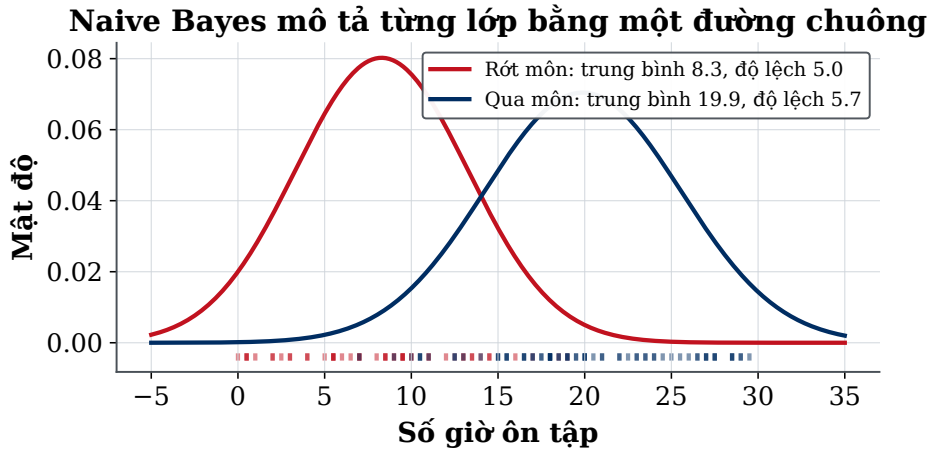
6. Naive Bayes cho số liệu liên tục

6.1. Đếm không còn dùng được

Đếm số lần một từ xuất hiện thì dễ, nhưng số giờ ôn tập là một số thực. Không ai đếm được *số lần giá trị 12.5 xuất hiện*, vì gần như mỗi sinh viên có một con số khác nhau.

Cách xử lý là giả định mỗi lớp có một **đường chuông** riêng, tức một phân phối chuẩn. Huấn

luyện khi đó chỉ gồm hai phép tính cho mỗi lớp và mỗi đặc trưng: tính trung bình và tính độ lệch chuẩn.



Hình 4: *Hai đường chuông mô tả số giờ ôn tập của nhóm rớt và nhóm qua. Các vạch nhỏ ở đáy là từng sinh viên thật.*

Biến thể nào cho kiểu dữ liệu nào

- `MultinomialNB` cho dữ liệu **đếm**, ví dụ số lần một từ xuất hiện.
- `GaussianNB` cho dữ liệu **liên tục**, ví dụ chiều cao, cân nặng, số giờ.
- `BernoulliNB` cho dữ liệu **có hoặc không**, ví dụ từ đó có mặt hay vắng mặt.

Chọn sai biến thể thì chương trình vẫn chạy và vẫn in ra một con số accuracy, nên đây là kiểu lỗi im lặng rất nguy hiểm.

6.2. So với hồi quy logistic trên cùng dữ liệu

```python
# -*- coding: utf-8 -*-
"""Bước 5: Naive Bayes cho số liệu liên tục, đặt cạnh hồi quy logistic."""

import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

df = pd.read_csv("data/sinh_vien.csv")
X = df[["gio_on", "diem_giua_ky"]]
y = df["qua_mon"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

# Hai mô hình, cùng dữ liệu, cùng cách chia, nên so sánh mới công bằng
for ten, mo_hinh in [("Naive Bayes     ", GaussianNB()),
                     ("Hoi quy logistic", LogisticRegression())]:
    mo_hinh.fit(X_train, y_train)
```

```
    acc = accuracy_score(y_test, mo_hinh.predict(X_test))
    print(f"{ten}: accuracy = {acc:.4f}")
print()

gnb = GaussianNB().fit(X_train, y_train)

# theta_ la trung binh, var_ la phuong sai, moi dong mot lop
print("Nhung gi Naive Bayes hoc duoc")
print(f"  {'':16s} {'gio_on':>10s} {'diem_giua_ky':>14s}")
for i, ten_lop in enumerate(["Lop 0 (rot) ", "Lop 1 (qua) "]):
    tb = gnb.theta_[i]
    dl = gnb.var_[i] ** 0.5
    print(f"  {ten_lop} trung binh {tb[0]:10.2f} {tb[1]:14.2f}")
    print(f"  {'':12s} do lech   {dl[0]:10.2f} {dl[1]:14.2f}")
print()

ty_le = ", ".join(f"{float(v):.4f}" for v in gnb.class_prior_)
print(f"Ty le tung lop trong tap hoc: {ty_le}")
print()

# Dự đoán cho vài sinh viên giả định
ban_moi = pd.DataFrame({"gio_on": [5, 13, 20, 28],
                        "diem_giua_ky": [4.0, 6.0, 7.0, 9.0]})
p = gnb.predict_proba(ban_moi)[:, 1]
print("Du doan cho bon ban moi:")
print(f"  {'gio on':>8s} {'diem GK':>9s} {'P(qua mon)':>12s}")
for (_, r), pi in zip(ban_moi.iterrows(), p):
    print(f"  {r.gio_on:8.0f} {r.diem_giua_ky:9.1f} {pi:12.4f}")
```

```
python code/d5_gaussian.py
```

```
Naive Bayes     : accuracy = 0.9000
Hoi quy logistic: accuracy = 0.9000

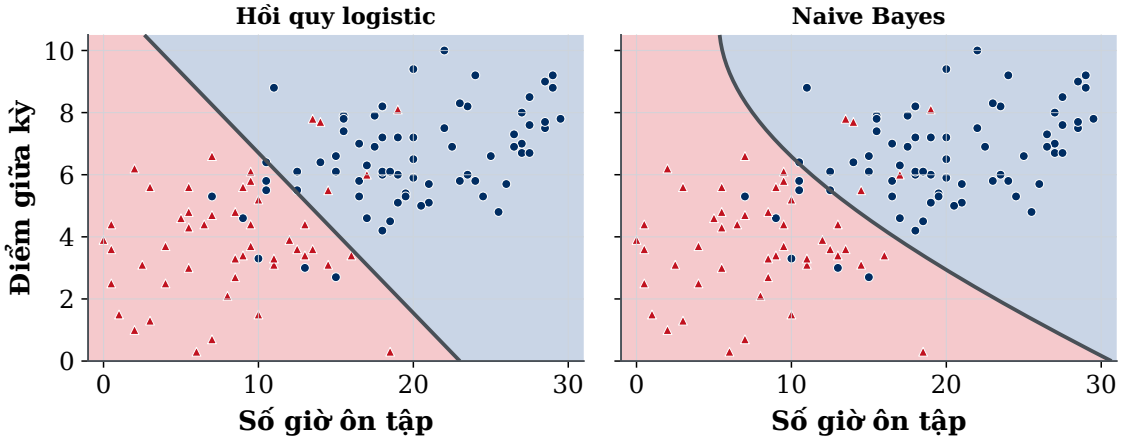
Nhung gi Naive Bayes hoc duoc
                      gio_on   diem_giua_ky
  Lop 0 (rot)  trung binh       8.18           3.99
               do lech         5.29           1.92
  Lop 1 (qua)  trung binh      20.03           6.70
               do lech         5.81           1.50

Ty le tung lop trong tap hoc: 0.4111, 0.5889

Du doan cho bon ban moi:
    gio on   diem GK   P(qua mon)
         5       4.0       0.0137
        13       6.0       0.6550
        20       7.0       0.9855
        28       9.0       0.9999
```

6.3. Đọc kết quả

Hai mô hình cho accuracy bằng nhau, cùng bằng 0.9000. Điều này không có nghĩa chúng là một, nó chỉ nói rằng trên bộ dữ liệu nhỏ này cả hai đều hợp.



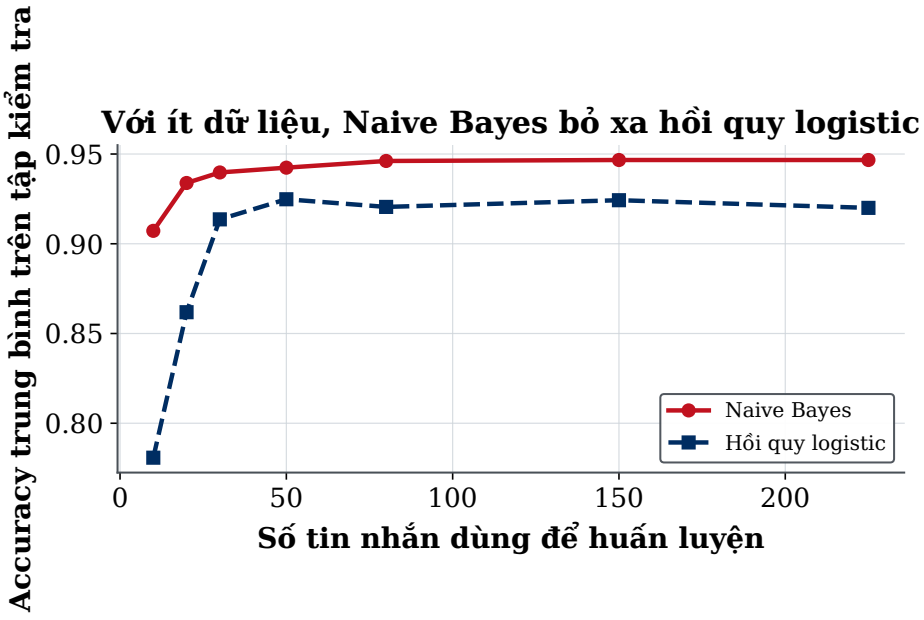
Hình 5: *Biên quyết định của hai mô hình trên cùng bộ dữ liệu. Hai đường gần như trùng nhau.*

Phần `theta_` và `var_` cho thấy Naive Bayes đã học được gì. Nhóm rớt có số giờ ôn trung bình 8.18 giờ và điểm giữa kỳ trung bình 3.99. Nhóm qua có 20.03 giờ và 6.70 điểm. Hai nhóm cách nhau rõ ở cả hai đặc trưng, nên bài toán này dễ với mọi mô hình.

Điểm khác nhau đáng nhớ giữa hai mô hình

Hồi quy logistic học **ranh giới** và không quan tâm mỗi lớp trông ra sao. Naive Bayes học **hình dạng từng lớp** rồi mới suy ra ranh giới.
Vì vậy Naive Bayes cho ta thêm thông tin: nó nói được nhóm rớt ôn trung bình bao nhiêu giờ. Hồi quy logistic không trả lời được câu đó.

6.4. Chỗ hai mô hình khác nhau rõ rệt



Hình 6: *Accuracy trung bình theo số tin nhắn dùng để huấn luyện, đo trên bài lọc tin rác.*

Trên bài lọc tin rác, nơi có tới 40 đặc trưng, sự khác nhau lộ rất rõ. Với chỉ 10 tin nhắn huấn luyện, Naive Bayes đạt 0.9072 còn hồi quy logistic chỉ đạt 0.7808. Khi dùng đủ 225 tin, khoảng cách thu hẹp còn 0.9467 so với 0.9200.

Vì sao Naive Bayes trụ tốt khi dữ liệu ít

Naive Bayes chỉ cần ước lượng 40 con số cho mỗi lớp, mỗi con số lấy từ một phép đếm đơn giản. Hồi quy logistic phải dò 40 trọng số cùng lúc sao cho chúng phối hợp với nhau, và với mười mẫu thì không đủ thông tin để dò.
Giả định ngây thơ tuy sai nhưng nó đóng vai trò một cái khung đỡ, giúp mô hình đứng vững khi dữ liệu còn mỏng. Đây là một ví dụ đẹp cho thấy một giả định sai vẫn có thể có ích.

Bảng 5: *Chọn mô hình nào cho tình huống nào.*

Tình huống	Naive Bayes	Hồi quy logistic
Dữ liệu ít, đặc trưng nhiều	Rất hợp	Dễ học vẹt
Dữ liệu nhiều	Thường thua một chút	Thường nhỉnh hơn
Đặc trưng liên quan chặt	Xác suất bị thổi phồng	Xử lý tốt hơn
Cần huấn luyện thật nhanh	Chỉ đếm một lượt	Phải dò nhiều vòng
Cần xác suất đáng tin	Không nên tin	Đáng tin hơn

Lời khuyên dùng được ngay

Với một bài phân loại mới, hãy chạy Naive Bayes trước tiên vì nó nhanh và gần như không có tham số nào cần chỉnh. Lấy kết quả đó làm mốc đối chứng, rồi mới thử những mô hình phức tạp hơn. Mô hình nào không vượt được mốc này thì chưa đáng dùng.

7. Bài tập

Sinh viên tạo thư mục `baitap03` nằm ngay trong thư mục bài thực hành. Vẫn chạy lệnh từ thư mục gốc, tức gõ `python baitap03/bai1.py`, và trong mã vẫn viết `data/tin_nhan.csv`. Bài nộp gồm tệp mã nguồn và ảnh chụp màn hình phần kết quả.

Bài tập 1: Bayes với ba nguyên nhân

Một lớp có ba nhóm sinh viên: 50 phần trăm học đều, 30 phần trăm học dồn, 20 phần trăm ít học. Tỷ lệ qua môn của ba nhóm lần lượt là 0.95, 0.70 và 0.30.
Gặp một bạn đã qua môn. Tính xác suất bạn đó thuộc từng nhóm. Ba con số cộng lại phải bằng 1, hãy dùng điều đó để tự kiểm tra.
Gợi ý, mở rộng hàm `bayes` trong `d1_bayes_tay.py` cho ba nhánh thay vì hai.

Bài tập 2: Từ nào tố cáo mạnh nhất

Huấn luyện `MultinomialNB` trên toàn bộ 300 tin nhắn. Dùng thuộc tính `feature_log_prob_` để tính hiệu giữa hai lớp cho từng từ, rồi in ra 5 từ tố cáo tin rác mạnh nhất và 5 từ cho thấy tin thường mạnh nhất.
Gợi ý, `feature_log_prob_` là một mảng hai dòng, dòng 0 cho lớp 0 và dòng 1 cho lớp 1. Lấy `mo_hinh.feature_log_prob_[1] - mo_hinh.feature_log_prob_[0]` rồi sắp xếp.

Bài tập 3: Đổi tham số làm trơn

Chạy lại bộ lọc với `alpha` lần lượt bằng 0.001, 0.1, 1.0, 10.0 và 100.0. Với mỗi giá trị, in ra accuracy, precision và recall trên tập kiểm tra.
Nhận xét một câu: alpha lớn quá thì chuyện gì xảy ra, và vì sao? Gợi ý cho phần giải thích, alpha rất lớn làm mọi xác suất từ xích lại gần nhau.

Bài tập 4: Tự viết bộ lọc không dùng thư viện

Dựa vào `d4_lam_tron.py`, viết một bộ lọc hoàn chỉnh cho toàn bộ 300 tin nhắn. Tự đếm từ, tự làm trơn, tự tính điểm cho hai lớp bằng **tổng loga** thay vì tích.
Chia dữ liệu bằng `train_test_split` với `random_state=17` rồi so accuracy của bộ lọc tự viết với `MultinomialNB`. Hai con số phải rất gần nhau. Nếu lệch nhiều thì có lỗi ở đâu đó, hãy tìm ra.

Bài tập 5: Chọn sai biến thể Naive Bayes

Chạy `MultinomialNB` trên bộ dữ liệu sinh viên với hai cột `gio_on` và `diem_giua_ky`. In ra accuracy rồi so với `GaussianNB` ở mục 6.
Chương trình có báo lỗi không? Con số accuracy ra bao nhiêu? Giải thích vì sao dùng `MultinomialNB` ở đây là sai về nguyên tắc, dù nó vẫn chạy.

Bài tập 6: Lớp hiếm và xác suất tiên nghiệm

Từ bộ tin nhắn, tạo một bộ mới mất cân bằng nặng: giữ toàn bộ 180 tin thường nhưng chỉ lấy 20 tin rác đầu tiên. Huấn luyện lại và chấm điểm.
In ra accuracy và recall của lớp rác. Nhận xét: accuracy có còn phản ánh đúng chất lượng bộ lọc không? Gợi ý, dùng `recall_score` với `pos_label=1`, và nhớ lại cái bẫy accuracy ở bài 2.

Tiêu chí chấm

Mã chạy được không lỗi, kết quả in ra đúng, có nhận xét bằng lời ở những bài yêu cầu, tên biến đặt có nghĩa. Bài 4 và bài 6 chấm nặng phần giải thích, vì đó là chỗ kiểm tra sinh viên có thật sự hiểu cơ chế hay chỉ gọi hàm.

8. Phụ lục A, bảng tra nhanh

Bảng 6: *Các hàm mới xuất hiện trong bài 3.*

Lệnh	Thư viện	Công dụng
`CountVectorizer`	scikit-learn	Biến một tập văn bản thành bảng đếm từ.
`.fit_transform`	scikit-learn	Lập từ điển rồi đếm luôn, dùng cho tập học.
`.transform`	scikit-learn	Chỉ đếm theo từ điển đã có, dùng cho dữ liệu mới.
`.get_feature_names_out`	scikit-learn	Lấy danh sách từ theo đúng thứ tự cột.
`.vocabulary_`	scikit-learn	Từ điển dạng ánh xạ từ sang chỉ số cột.
`MultinomialNB`	scikit-learn	Naive Bayes cho dữ liệu đếm.
`GaussianNB`	scikit-learn	Naive Bayes cho dữ liệu liên tục.
`BernoulliNB`	scikit-learn	Naive Bayes cho dữ liệu có hoặc không.
`alpha`	scikit-learn	Hệ số làm trơn Laplace, mặc định bằng 1.
`.feature_log_prob_`	scikit-learn	Loga xác suất của từng từ trong từng lớp.
`.theta_`	scikit-learn	Trung bình của từng đặc trưng theo từng lớp.
`.var_`	scikit-learn	Phương sai của từng đặc trưng theo từng lớp.
`.class_prior_`	scikit-learn	Xác suất tiên nghiệm của từng lớp.
`.toarray()`	scipy	Đổi ma trận thưa thành mảng thường để nhìn.
`Counter`	collections	Đếm số lần xuất hiện, dùng khi tự cài đặt.

Bảng 7: *Ký hiệu và thuật ngữ của bài 3.*

Ký hiệu hoặc từ	Ý nghĩa
$\mathbb{P}(A \mid B)$	Xác suất của A khi đã biết B xảy ra.
Tiên nghiệm	Niềm tin về một lớp trước khi nhìn vào dữ liệu, chính là tỷ lệ lớp đó.
Khả năng	Mức độ mà một lớp có thể sinh ra bằng chứng đang xét.
Hậu nghiệm	Niềm tin về lớp sau khi đã nhìn bằng chứng, đây là thứ ta muốn.
α	Hệ số làm trơn Laplace.
V	Số từ trong từ điển.
Túi từ	Cách biểu diễn văn bản bằng số lần xuất hiện của từng từ.
Giả định ngây thơ	Coi các đặc trưng độc lập với nhau khi đã biết lớp.
Tràn dưới	Tích của nhiều số nhỏ bị máy làm tròn thành 0.

Bảng 8: *Các con số kết quả chính của bài thực hành.*

Đại lượng	Giá trị
Hậu nghiệm của ví dụ xét nghiệm	0.5000
Ví dụ hai dây chuyền, khả năng từ dây B	0.5556
Kho tin nhắn	300 tin, 120 rác và 180 thường
Số từ trong từ điển	40
Chia dữ liệu	225 để học, 75 để kiểm tra
Ma trận nhầm lẫn của bộ lọc	TP $= 26$, TN $= 45$, FP $= 0$, FN $= 4$
Accuracy, Precision, Recall	0.9467, 1.0000, 0.8667
Naive Bayes và logistic trên dữ liệu sinh viên	cùng bằng 0.9000
Với 10 tin huấn luyện	Naive Bayes 0.9072, logistic 0.7808

9. Phụ lục B, lỗi thường gặp

Bảng 9: *Các lỗi hay gặp riêng của bài Naive Bayes.*

Thông báo lỗi hoặc hiện tượng	Nguyên nhân và cách xử lý
`ValueError: Negative values in data`	Đưa dữ liệu có giá trị âm vào `MultinomialNB`. Biến thể này chỉ nhận số đếm không âm, hãy đổi sang `GaussianNB`.
Từ điển thiếu mất nhiều từ	`CountVectorizer` mặc định bỏ từ một ký tự và tách chữ theo quy tắc riêng. Truyền `token_pattern=r"(?u)\S+"` để tách bằng khoảng trắng.
Dùng `fit_transform` cho tập kiểm tra	Làm vậy sẽ lập lại từ điển mới và số cột không khớp mô hình. Tập học dùng `fit_transform`, tập kiểm tra và dữ liệu mới chỉ dùng `transform`.
`ValueError`, số cột không khớp	Cùng nguyên nhân như trên, hoặc dùng nhầm hai đối tượng `CountVectorizer` khác nhau.
Mọi xác suất in ra đều bằng 0	Đang nhân trực tiếp nhiều xác suất nhỏ nên bị tràn dưới. Hãy cộng loga thay vì nhân, dùng `np.log`.
Mô hình luôn đoán một lớp duy nhất	Hai lớp lệch nhau quá nhiều nên xác suất tiên nghiệm lấn át. Kiểm tra `value_counts()` của cột nhãn, và đừng chỉ nhìn accuracy.
`MemoryError` khi gọi `.toarray()`	Bảng đếm là ma trận thưa, đổi sang mảng thường sẽ nở ra rất lớn. Chỉ gọi `.toarray()` trên một phần nhỏ để xem, còn mô hình nhận thẳng ma trận thưa được.
Accuracy rất cao nhưng bộ lọc vô dụng	Lớp rác quá hiếm trong dữ liệu. Hãy xem recall của riêng lớp rác bằng `recall_score` với `pos_label=1`.
Kết quả khác tài liệu	Quên `random_state=17` hoặc quên `stratify=y` khi chia dữ liệu.

Một lời khuyên chung

Naive Bayes rất ít khi báo lỗi, vì nó chỉ đếm và nhân. Vì vậy lỗi của bài này hầu hết là lỗi **im lặng**: chương trình chạy trơn tru nhưng kết quả vô nghĩa.
Ba câu tự hỏi trước khi tin vào kết quả. Mình đã chọn đúng biến thể Naive Bayes cho kiểu dữ liệu chưa? Hai lớp có lệch nhau quá đáng không? Mình có đang dùng con số xác suất của Naive Bayes như một xác suất thật không?